ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my project guide, **Dr. Khyati Rami** and **Prof. Nairutvi Mistry** , for their valuable guidance, support, and encouragement throughout the development of my project, **"AI-Based Document Authenticity Detection System".** Their insights and suggestions greatly contributed to the successful completion of this work.I would also like to thank the **Department of Computer Application, SAL Institute of Technology and Engineering Research, Ahmedabad,** for providing the necessary resources and a supportive environment to carry out this project.This project has been developed independently by me, and it has been a valuable learning experience that enhanced my understanding of artificial intelligence, machine learning, and real-world problem-solving.Finally, I would like to acknowledge the authors, researchers, and online resources whose work in the field of AI and machine learning provided valuable references and guidance during the development of this project.

Name of the student

Panchal Dax Hareshkuumar

ABSTRACT

This project presents an AI-based system for detecting AI-generated content in both textual and visual formats. The system allows users to upload documents (PDF, DOCX, TXT) and images, which are then analyzed using pretrained machine learning models. For text detection, a transformer-based model (RoBERTa) is used to classify sentences as AI-generated or human-written. For image analysis, a vision-language model (CLIP) combined with image processing techniques is used to identify AI-generated images and detect visual artifacts.

The system performs sentence-level classification and provides probability scores for better interpretability. Additionally, it integrates a hybrid approach by combining deep learning models with rule-based analysis to improve detection reliability. A secure authentication mechanism is implemented to control access to report downloads. The system generates structured PDF reports containing analysis results, making it suitable for academic and content verification purposes.

Keywords:

AI Content Détection, RoBERTa, CLIP, Text Analysis, Image Analysis, Machine Learning, Document Processing, Explainable Results

LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
ML	Machine Learning
NLP	Natural Language Processing
PDF	Portable Document Format
DOCX	Microsoft Word Document
TXT	Plain Text File
UI	User Interface
API	Application Programming Interface
OCR	Optical Character Recognition
CLIP	Contrastive Language–Image Pretraining
GPU	Graphics Processing Unit

LIST OF FIGURES

Sr. No	Figure Name	Page No.
1	System Flow Diagram …….…….……	07
2	Use Case Diagram ……..……..	08
3	Class Diagram……	09
4	Interaction Diagram……...……..…….	10
5	Activity Diagram……...…….	11
6	Screen Short……..……..	17
7	Burn chart……..……	33

INDEX

ACKNOWLEDGEMENTS.....i
ABSTRACT.....ii
LIST OF ABBREVIATIONS.....iii
LIST OF FIGURES.....iv
1: Introduction.....1
1.1Existing System.....1
1.2 Objective.....1
1.3 Problem Statement.....1
1.4 Core Components.....2
1.5 Project Profile.....3
1.6 Scope of the work.....3
1.7 Advantages and Limitations.....4
2: Requirement Determination & Analysis.....5
2.1 Requirement Determination.....5
2.2 Targeted Users.....6
3: System Design.....7
3.1 System Flow Diagram.....7
3.2 Use Case Diagram.....8
3.3 Class Diagram.....9
3.4 Interaction Diagram.....10
3.5 Activity Diagram.....11
3.6 Data Dictionary.....14
4: Development Progress / Implementation.....17
4.1 Coding standards.....17
4.2 Screen Shots.....19
5: Agile Documentation.....28
5.1 Agile Project Charter 28
5.2 Agile Roadmap / Schedule.....29
5.3 Agile Project Plan.....30
5.4 Agile User Story.....30
5.5 Agile Release Plan.....31
5.6 Agile Sprint Backlog.....31
5.7 Agile Test Plan.....32
5.8 Earned-value and burn charts.....32
6: Proposed Enhancements.....34
7 : Conclusion.....35
8: Bibliography.....36
9.Progress report.....37
10.Feedback.....42

Chapter 1: Introduction

1.1Existing System

Turnitin – Primarily focused on plagiarism detection with limited AI detection capability.**Copyleaks** – Provides AI detection but lacks deep transparency in model explanation.Other standalone AI detectors – Provide probability scores but often do not perform sentence-level analysis or image detection.Most existing systems focus only on textual plagiarism rather than AI-authorship detection and rarely combine text and image analysis.

1.2 Objective

To develop a system that can classify text as AI-generated or human-written using pretrained transformer-based models such as RoBERTa. To perform sentence-level analysis for detailed and interpretable results. To detect AI-generated images using pretrained models such as CLIP along with image processing techniques. To provide probability-based outputs indicating AI and human likelihood. To generate structured PDF reports containing analysis results. To implement secure user authentication for controlled access to reports. To design a hybrid system combining machine learning models with rule-based analysis for improved reliability.

1.3 Problem Statement

AI-generated assignments and essays are increasingly being submitted as original work, raising concerns about authenticity and academic integrity. Identifying AI-generated content manually is difficult and time-consuming, especially with the increasing sophistication of modern AI tools. Many existing tools lack sentence-level classification, making it hard to analyze content in detail and understand specific AI-generated segments. There is no unified system that can detect both AI-generated text and AI-generated images within a single platform.

1.4 Core Components

1.User Interface (Frontend)

Allows users to upload documents and images

Displays analysis results with highlighting and probabilities

2.Backend Server (Django)

Handles requests and processing logic

Connects Frontend with AI models and database

3.Text Detection Module

Uses pretrained models like RoBERTa

Performs sentence-level AI vs Human classification

4.Image Detection Module

Uses pretrained model CLIP

Detects AI-generated images and calculates probabilities

5.Image Processing (OpenCV)

Detects artifacts like blur, edge density, and texture

Supports AI detection with heuristic analysis

6.Database System

Stores user data, documents, and analysis results

Maintains report history

7.Report Generation Module

Generates PDF reports using ReportLab

Includes results, probabilities, and summary

8.Authentication System

Manages user login and registration

Restricts report downloads to authorized users

1.5 Project Profile

1.Title :

AI-Based Document Authenticity Detection System

2.Category :

Artificial Intelligence,Natural Language Processing (NLP),Computer Vision, Web Application Development

3.Duration :

18 December 2024 – 17 April 2025

4.Member :

Panchal Dax Hareshkumar

5.Tools & Technologies:

Backend : Python , Django

ML: PytTorch, HuggingFace Transformers, Sentence-Transformers, Scikit-learn

Frontend: HTML, CSS3, Bootstrap, JavaScript, jQuery

Database: SQLlite

Libraries : Pillow (Image Processing), ReportLab (PDF generation)

1.6 Scope of the work

The system focuses on detecting AI-generated content in both text and images using pretrained machine learning models. It supports document upload (PDF, DOCX, TXT) and performs sentence-level text classification along with probability-based results. The image analysis module detects AI-generated images and visual artifacts using advanced processing techniques.The system generates structured PDF reports and includes secure authentication for controlled access. It is primarily designed for academic and content verification purposes. However, it is limited to supported file formats and mainly focuses on English language content, and the results may not be 100% accurate in all cases.

1.7 Advantages and Limitations

1.7.1 Advantages:

Multi-modal detection (Text + Image).

Sentence-level explainable AI classification.

Secure user authentication for report downloads.

Scalable backend architecture.

Research-oriented design suitable for academic environments.

Modular ML integration for future expansion.

This makes the system more transparent and reliable compared to existing tools.

1.7.2 Limitations

No integrated multi-modal detection (text + image in one platform).

Limited sentence-wise highlighting.

Lack of AI-style similarity explanation.

Report download not always secured with authentication.

Black-box output without technical transparency.

High dependency on internet-based APIs.

These limitations reduce trust and analytical clarity.

Chapter 2: Requirement Determination & Analysis

2.1 Requirement Determination

Requirement determination involves identifying the essential features and functionalities that the system must provide to meet user needs.

2.1.1 Functional Requirements

User registration and login functionality

Upload documents (PDF, DOCX, TXT)

Extract and preprocess text from documents

Perform sentence-level AI detection using pretrained models (RoBERTa)

Upload and analyze images

Detect AI-generated images using pretrained models (CLIP + OpenCV)

Display AI and Human probability scores

Highlight classification results (AI / Human / Mixed)

Generate PDF reports of analysis

Allow report download only for authenticated users

2.1.2 Non-Functional Requirements

Performance: Fast processing of text and image data

Security: Authentication and access control for reports

Usability: Simple and user-friendly interface

Reliability: Proper error handling and stable execution

Scalability: Ability to handle multiple users and requests

File Constraint: Maximum file upload size limited to 10 MB

2.2 Targeted Users

Students: To verify originality of assignments

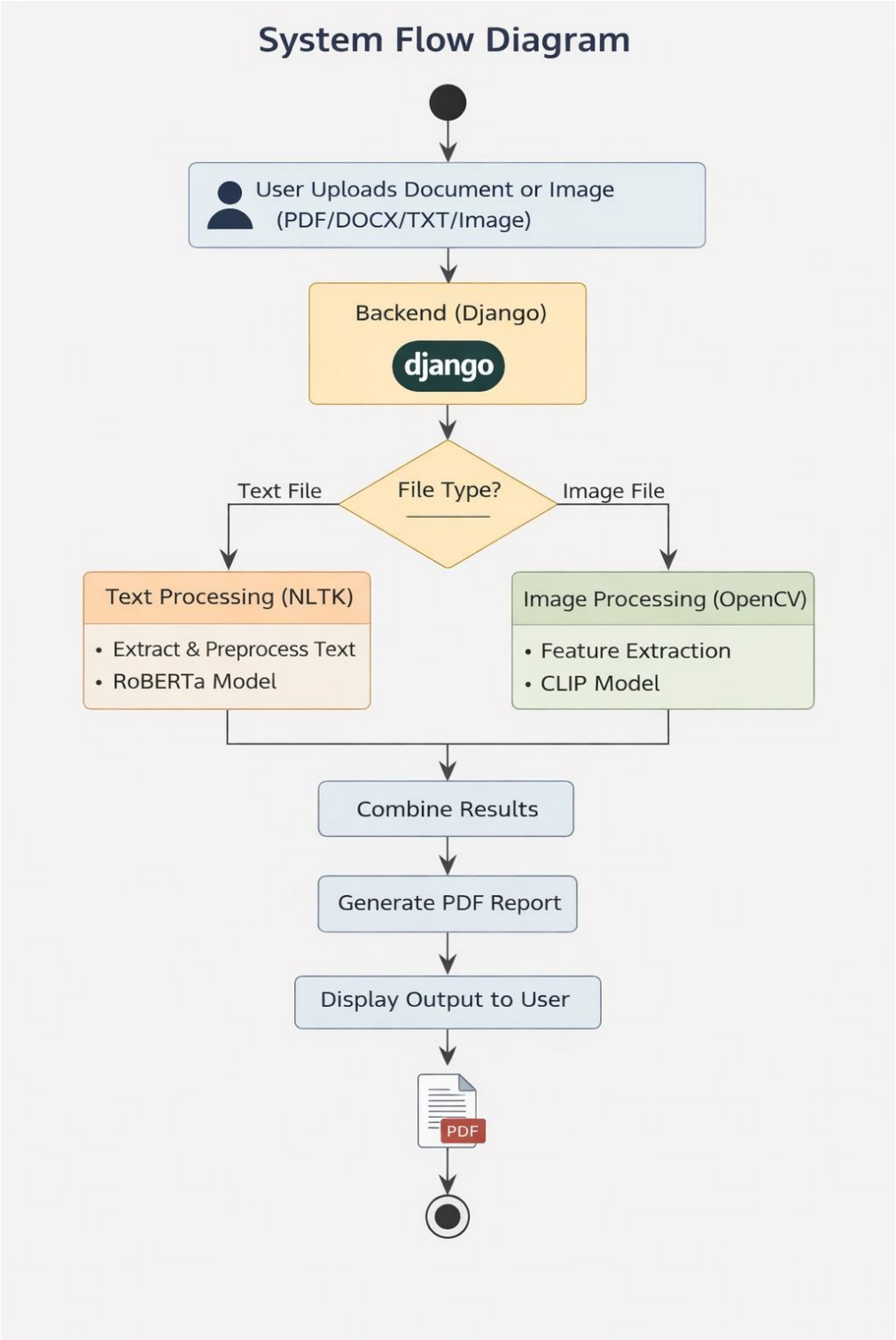
Teachers / Professors: To detect AI-generated submissions

Academic Institutions: To maintain academic integrity

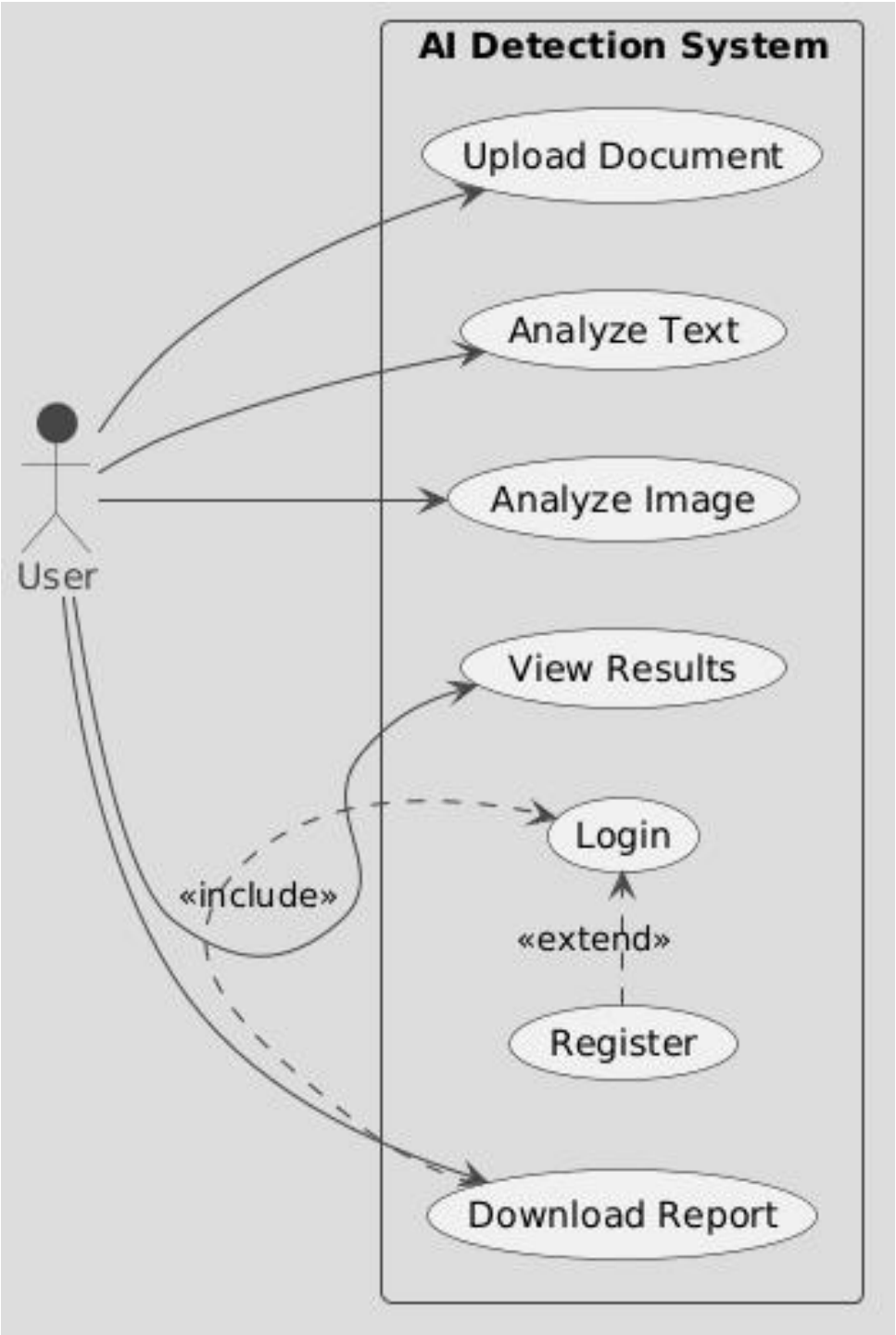
Content Reviewers: To analyze authenticity of text and images

Chapter 3: System Design

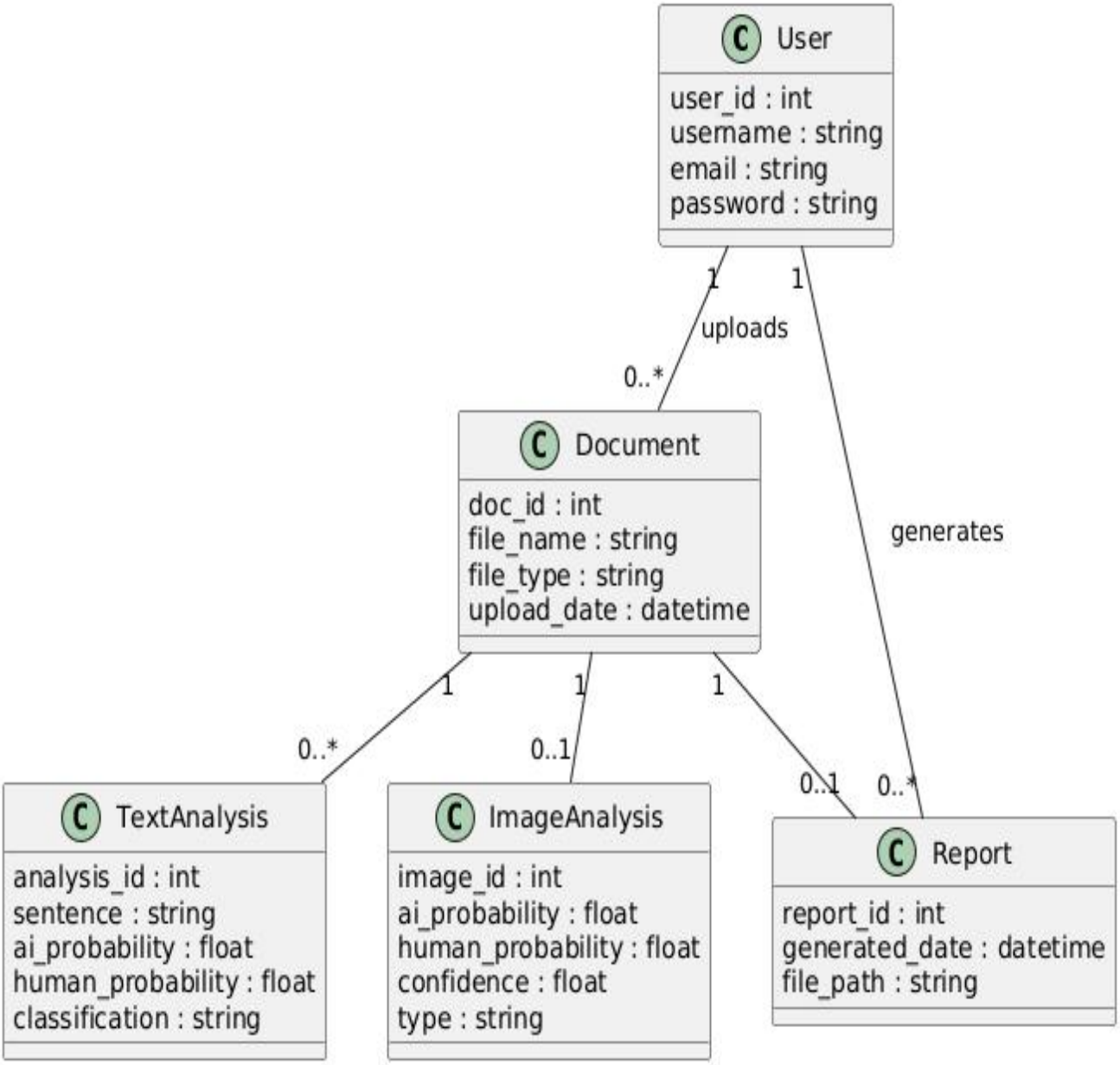
3.1 System Flow Diagram



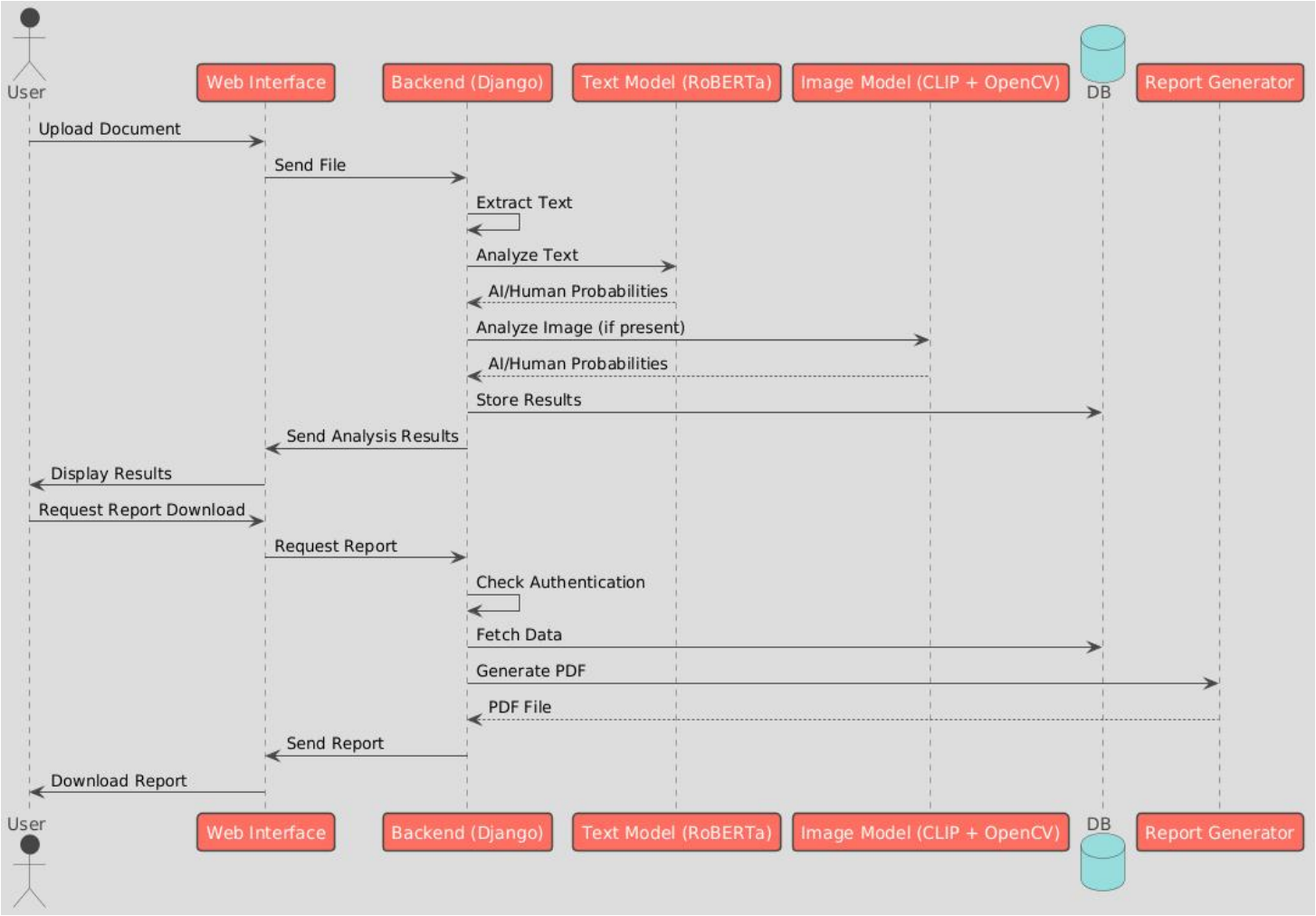
3.2 Use Case Diagram



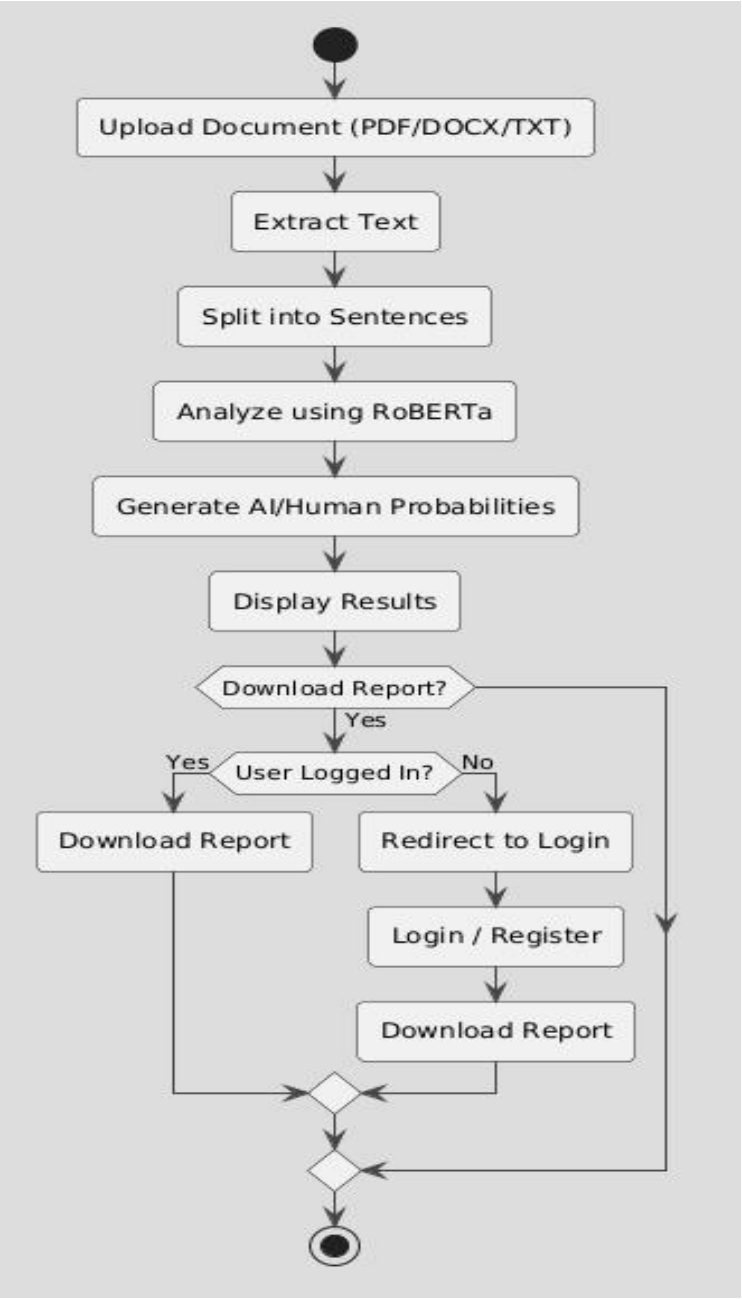
3.3 Class Diagram



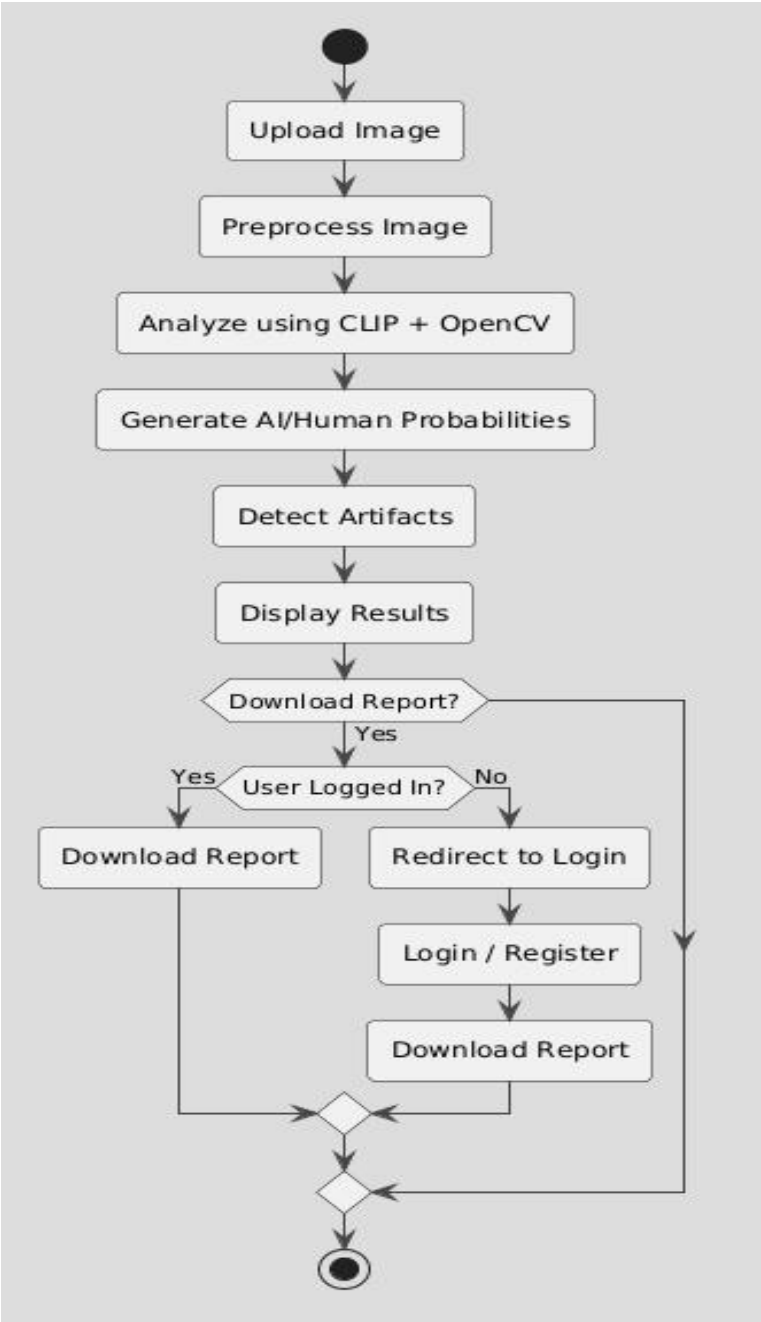
3.4 Interaction Diagram



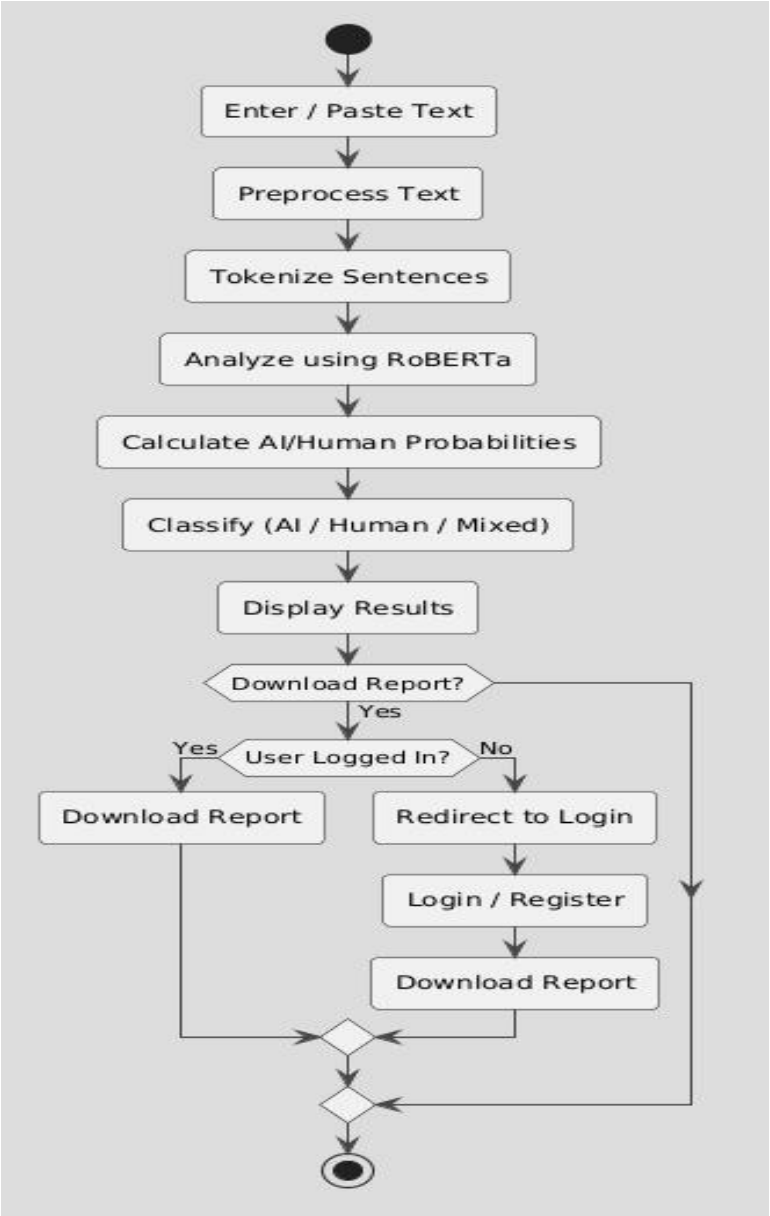
3.5 Activity Diagram (Document Upload)



3.5.1Activity Diagram (Image Upload)



3.5.2Activity Diagram (Text Analyzer)



3.6 Data Dictionary

3.7.1 User Table

No	Field Name	Type	Description
1	user_id	Integer	Unique identifier for user
2	username	String	Name of the user
3	email	String	User email address
4	password	String	Encrypted password

3.7.2 Document Table

No	Field Name	Type	Description
1	doc_id	Integer	Unique document ID
2	file_name	String	Name of uploaded file
3	file_type	String	File format (PDF/DOCX/TXT)
4	update_date	DateTime	Upload timestamp
5	user_id	Integer (FK)	Reference to User

3.7.3 TextAnalysis Table

No	Field Name	Type	Description
1	analysis_id	Integer	Unique analysis ID
2	sentence	String	Individual sentence
3	ai_probability	Float	Probability of AI-generated
4	human_probability	Float	Probability of human-written
5	classification	String	AI / Human / Mixed
6	doc_id	Integer	Reference to Document

3.7.4 ImageAnalysis Table

No	Field Name	Type	Description
1	image_id	Integer	Unique image analysis ID
2	ai_probability	Float	AI-generated probability
3	human_probability	Float	Human probability
4	Confidence	Float	Confidence score
5	type	String	Generation type (GAN/Diffusion)
6	doc_id	Integer (FK)	Reference to Document

3.7.5 Report Table

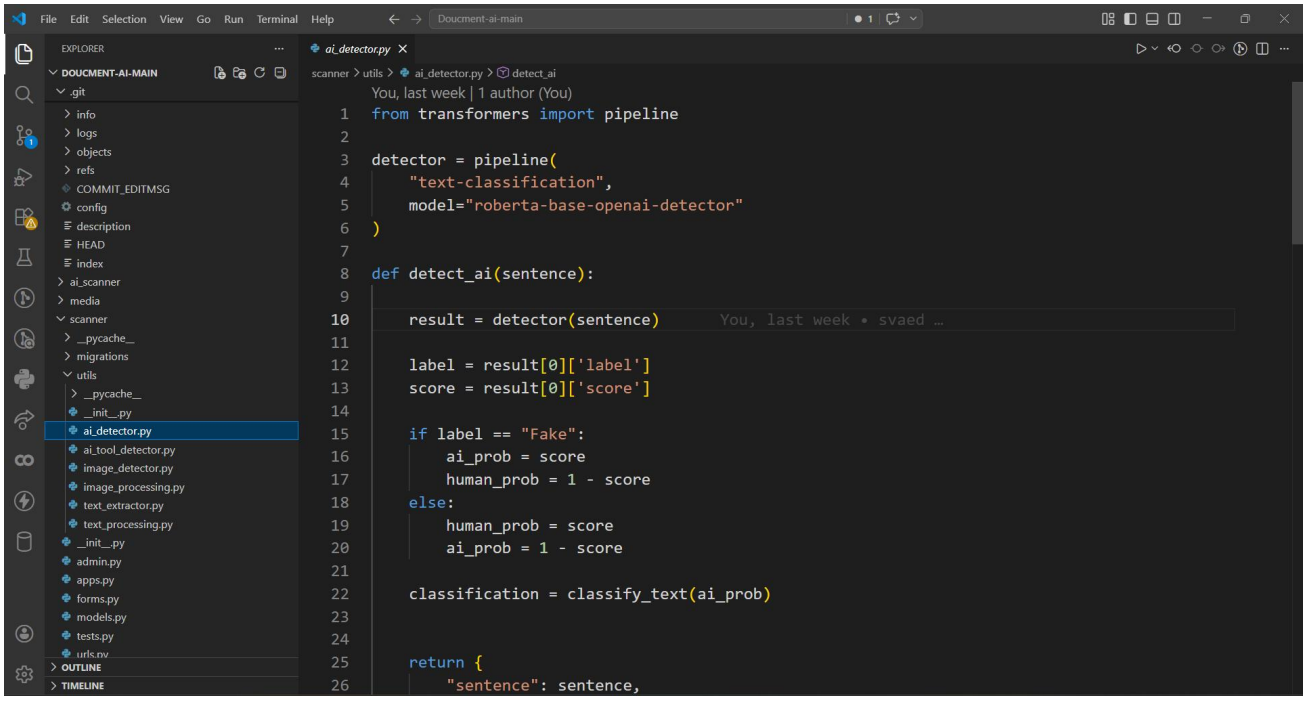
No	Field Name	Type	Description
1	report_id	Integer	Unique report ID
2	generated_date	DateTime	Report generation time
3	file_path	String	Stored PDF path
4	user_id	Integer (FK)	Reference to User
5	doc_id	Integer (FK)	Reference to Document

Chapter 4: Development Progress / Implementation

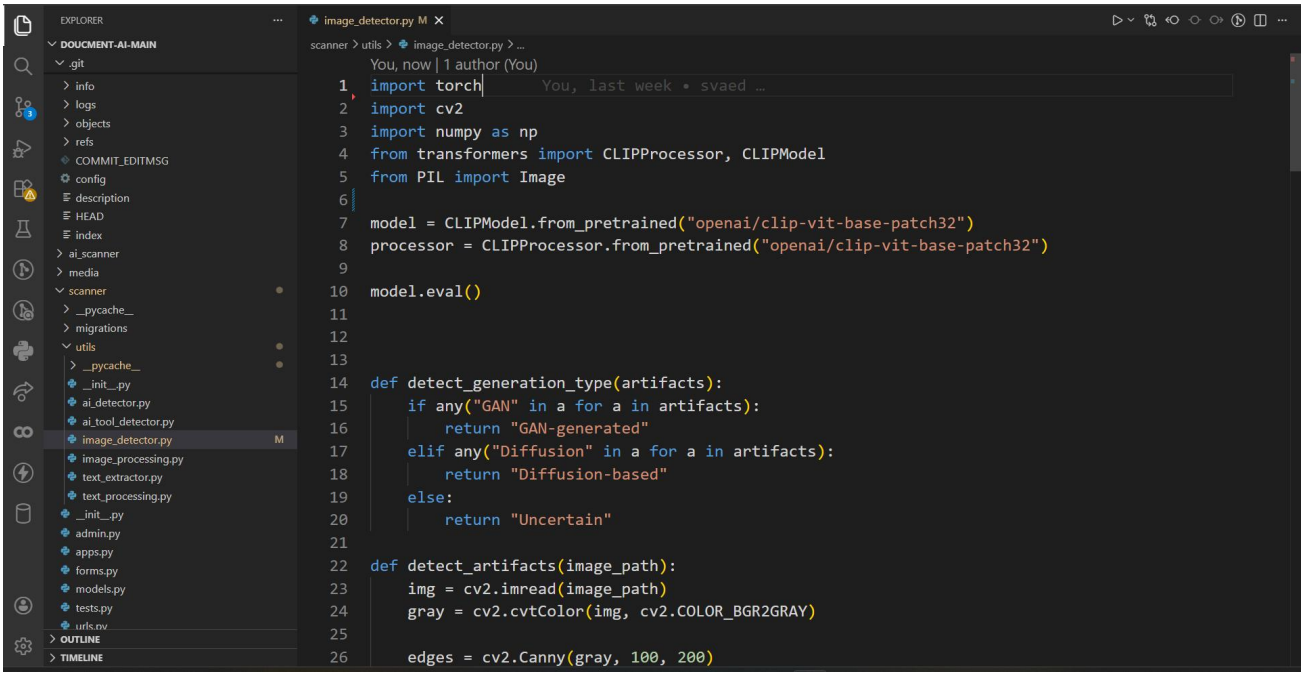
4.1 Coding standards

The project follows standard coding practices to ensure maintainability, readability, and reliability.It uses meaningful naming conventions and a modular structure for better organization and scalability. Proper formatting and indentation improve code clarity and consistency.Error handling and reusable functions are implemented to enhance efficiency and prevent failures.Security measures like authentication and restricted access are applied to protect data.

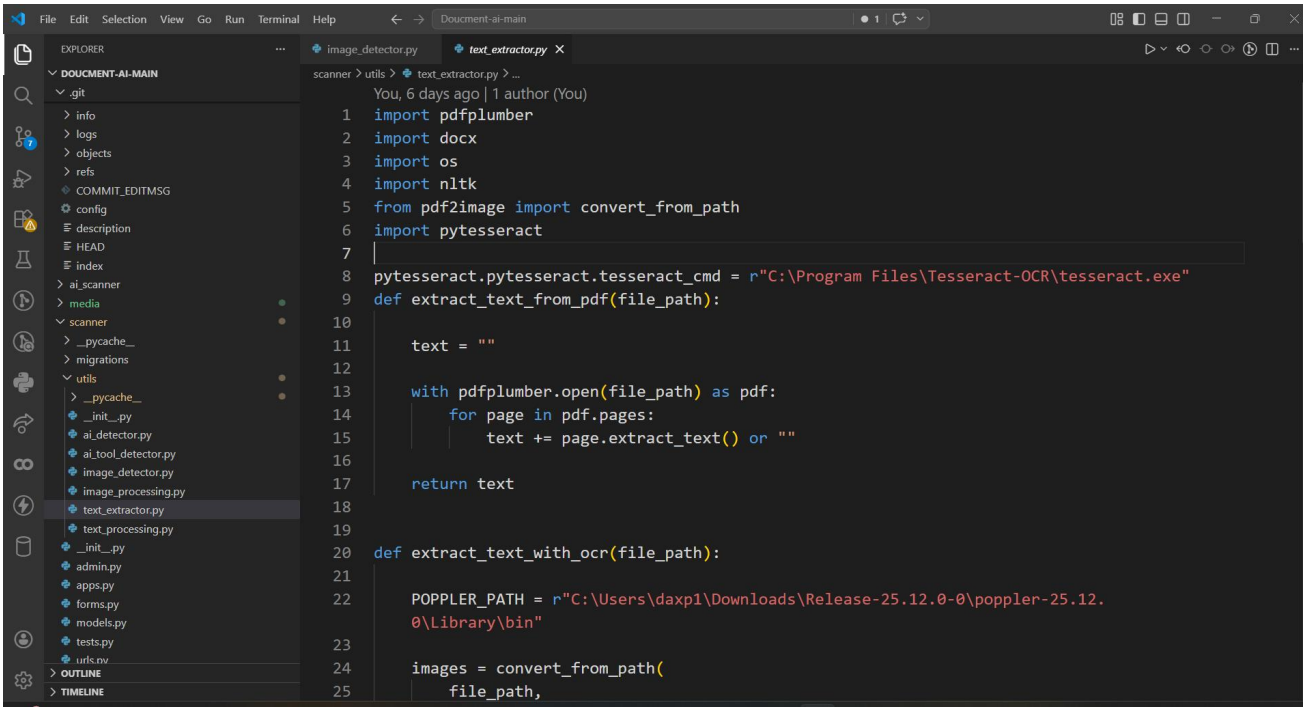
4.1.1 Coding Screenshots



4.1.2 Coding Screenshots

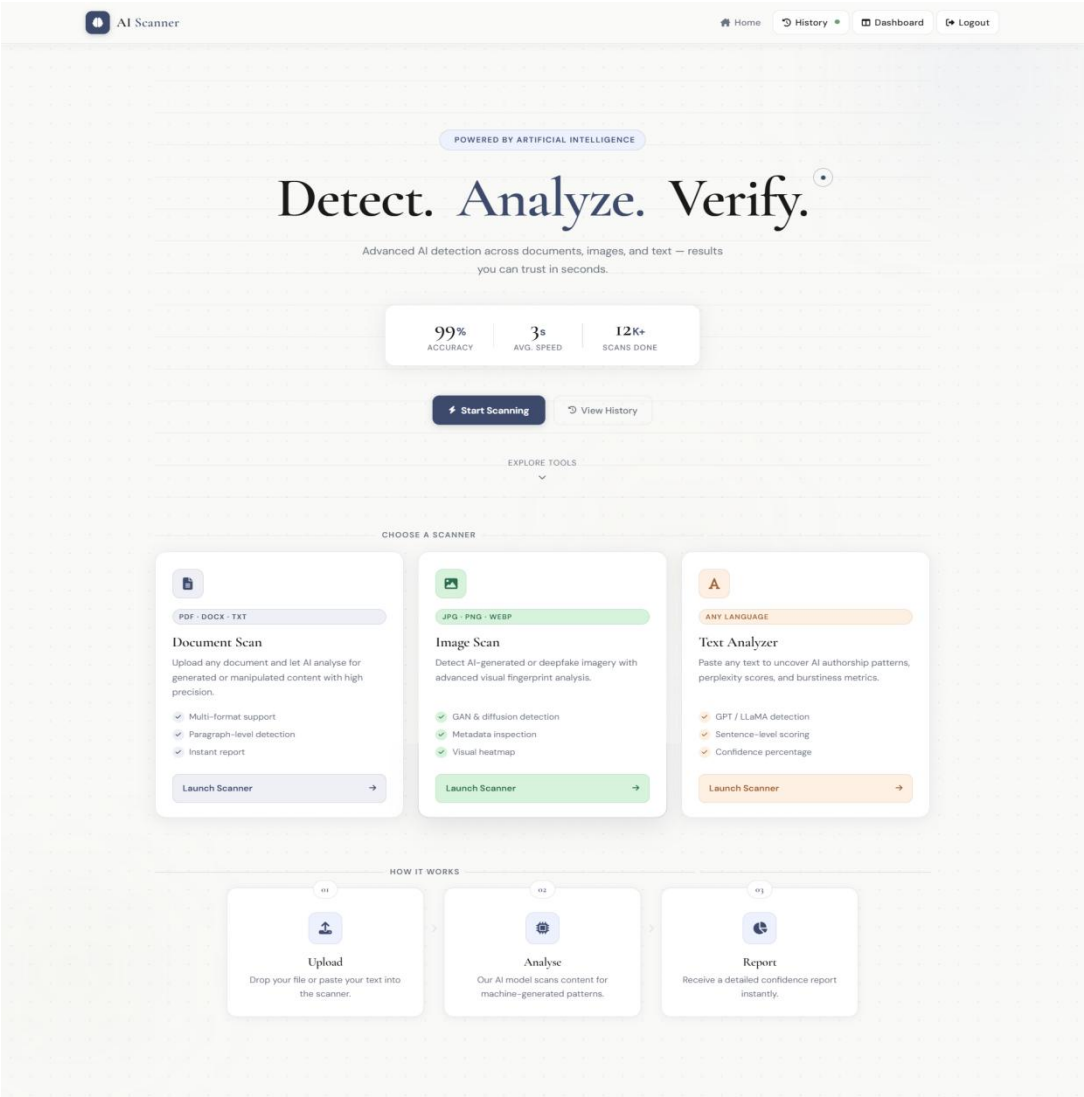


4.1.3 Coding Screenshots

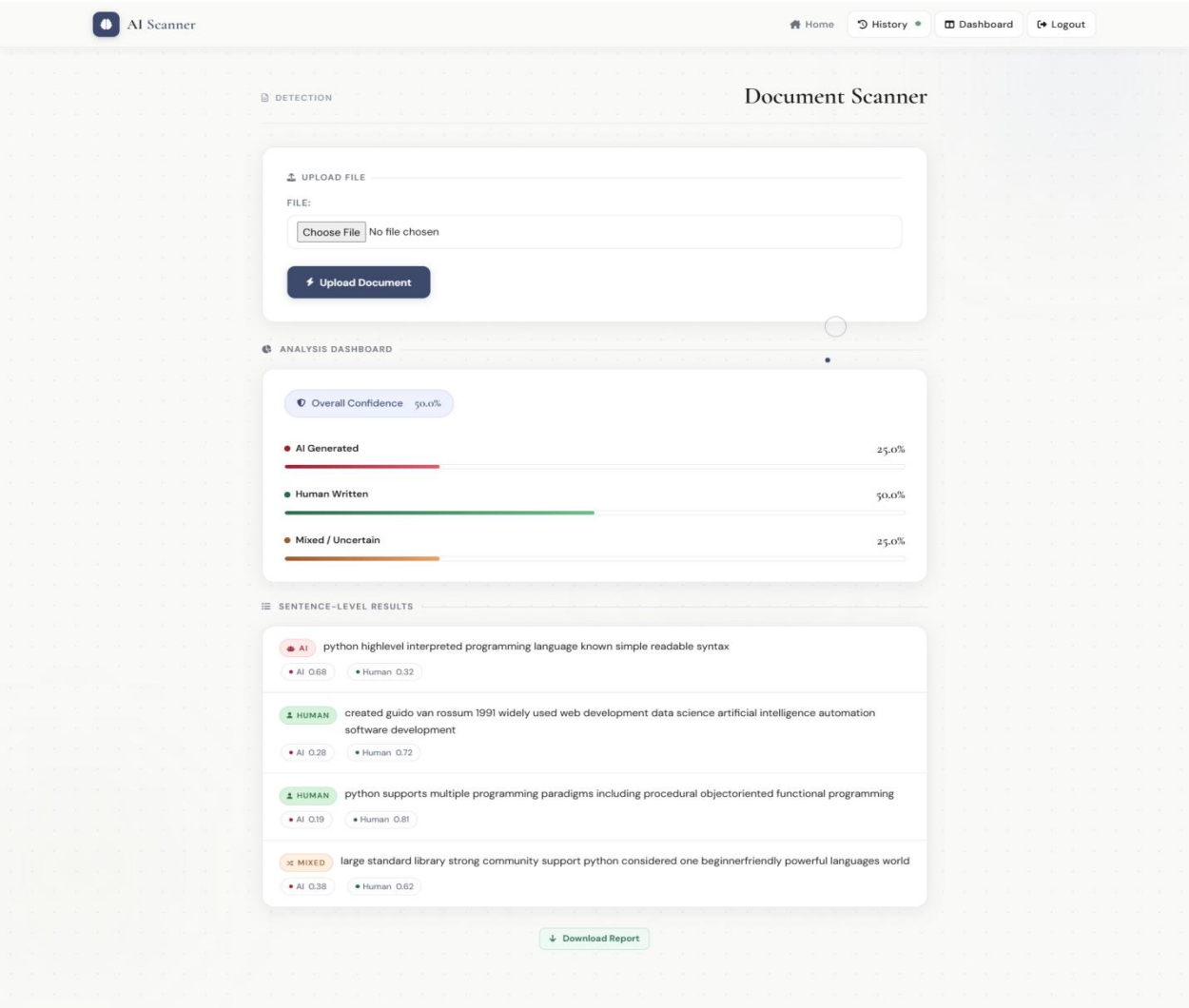


4.2 Screen Shots

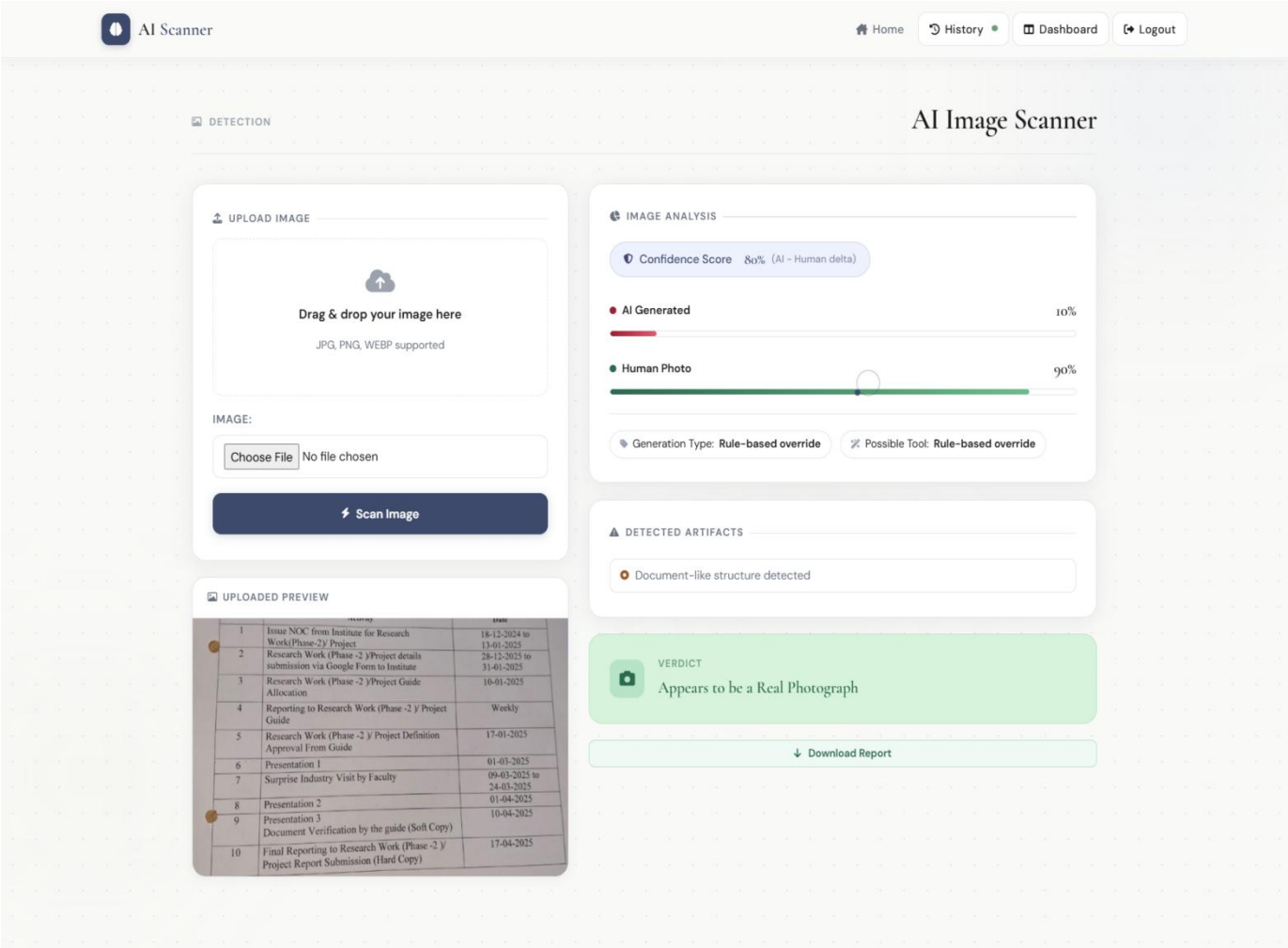
4.2.1 Home



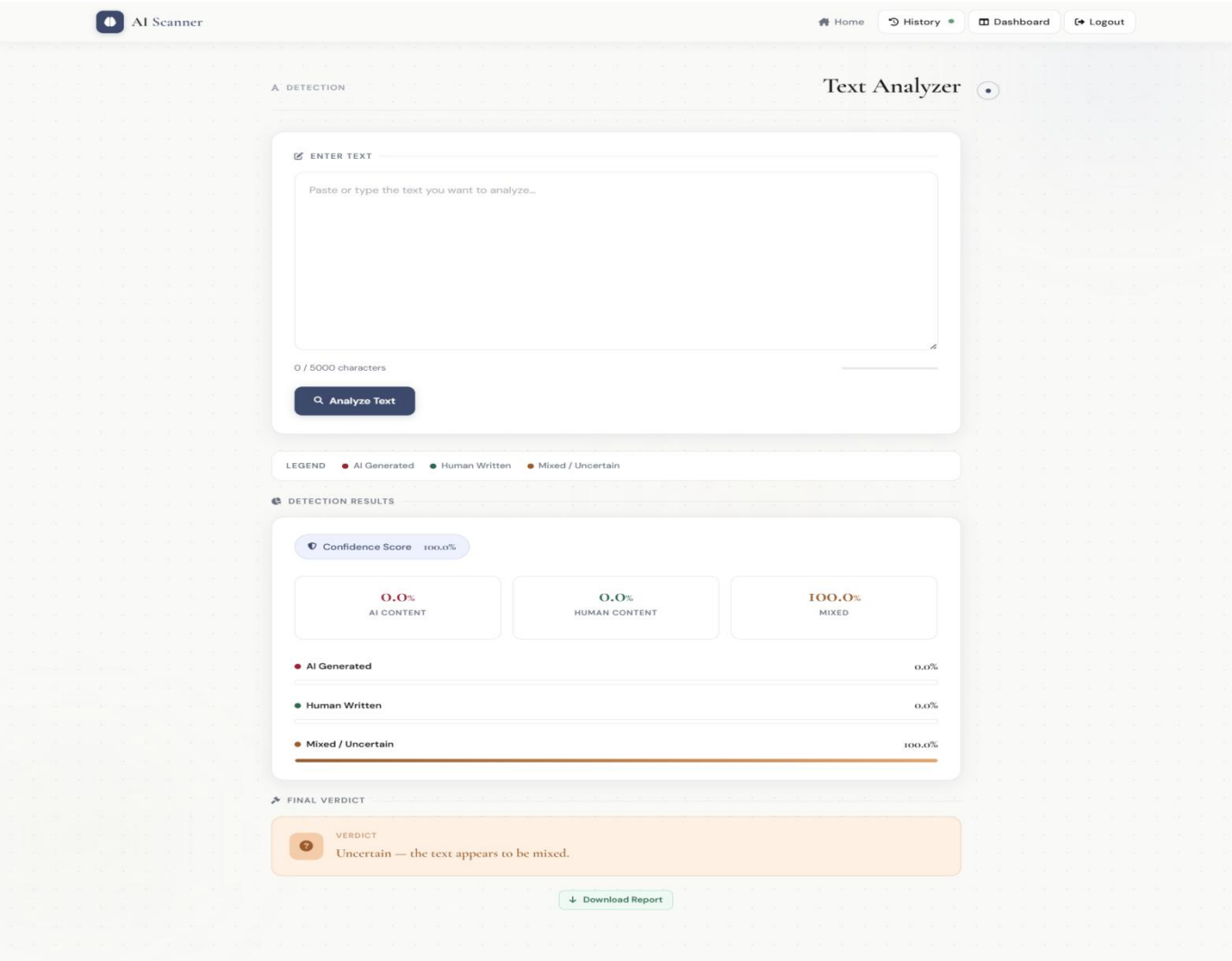
4.2.2 Document Scanner



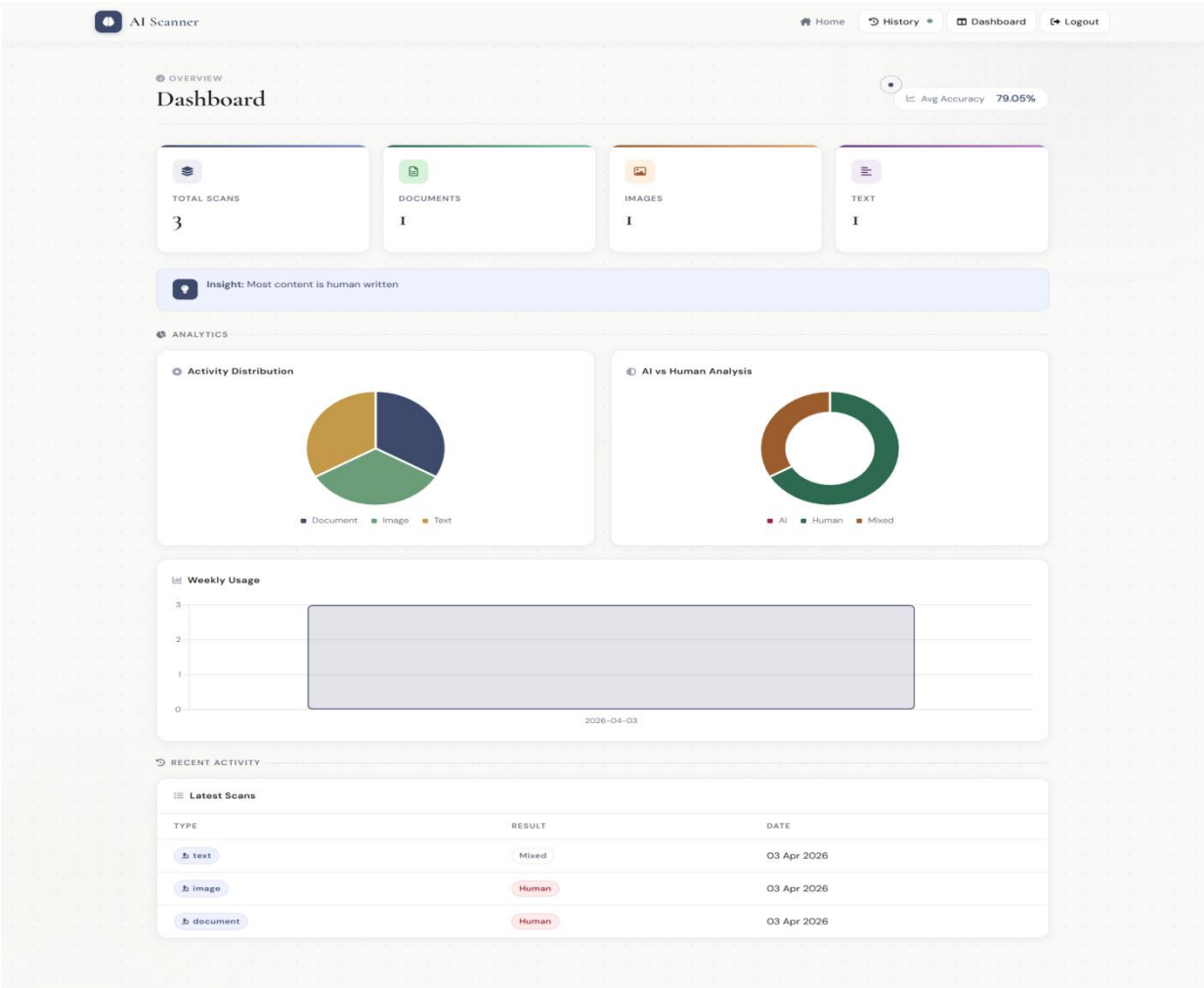
4.2.3 Image Scanner



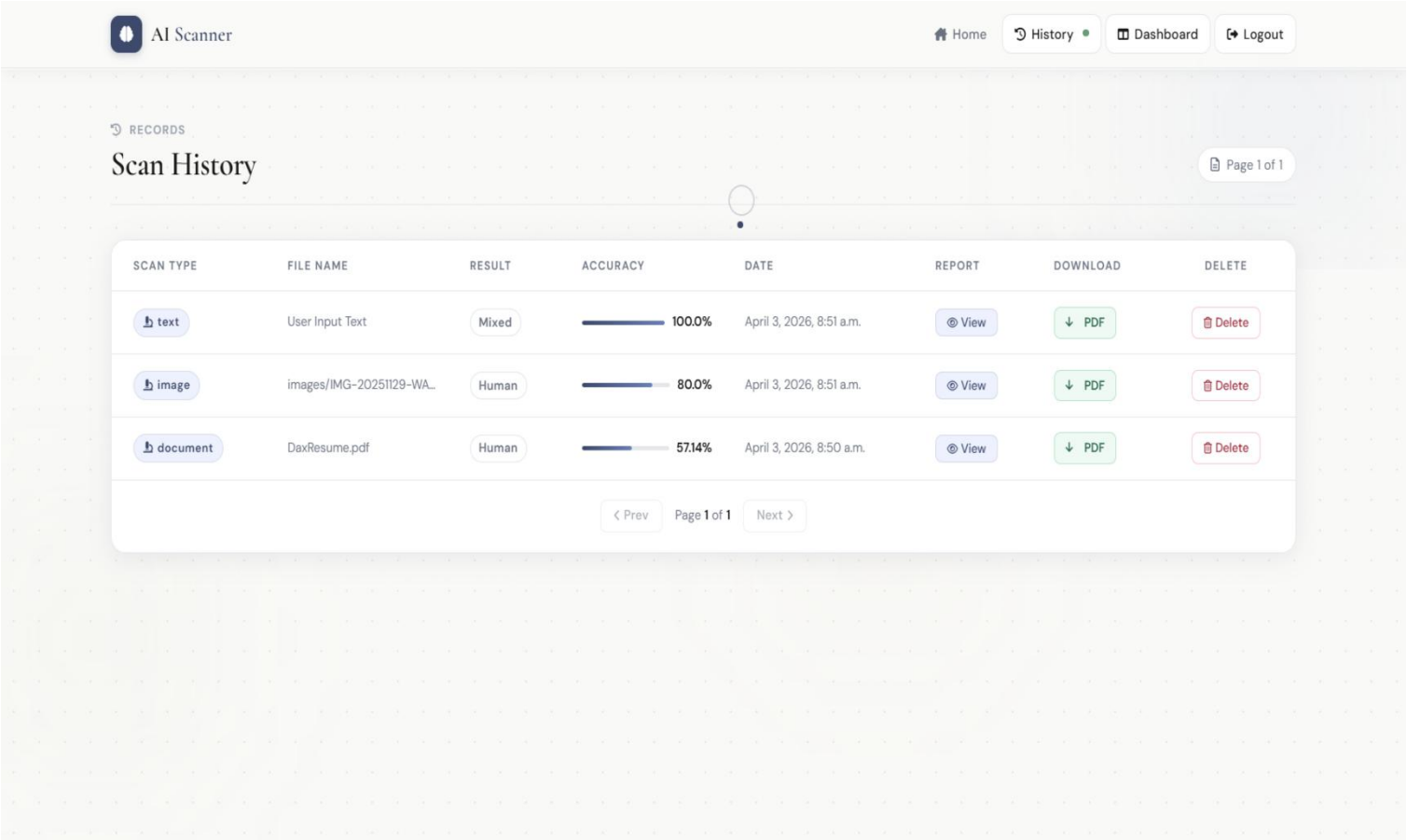
4.2.4 Text analyzer



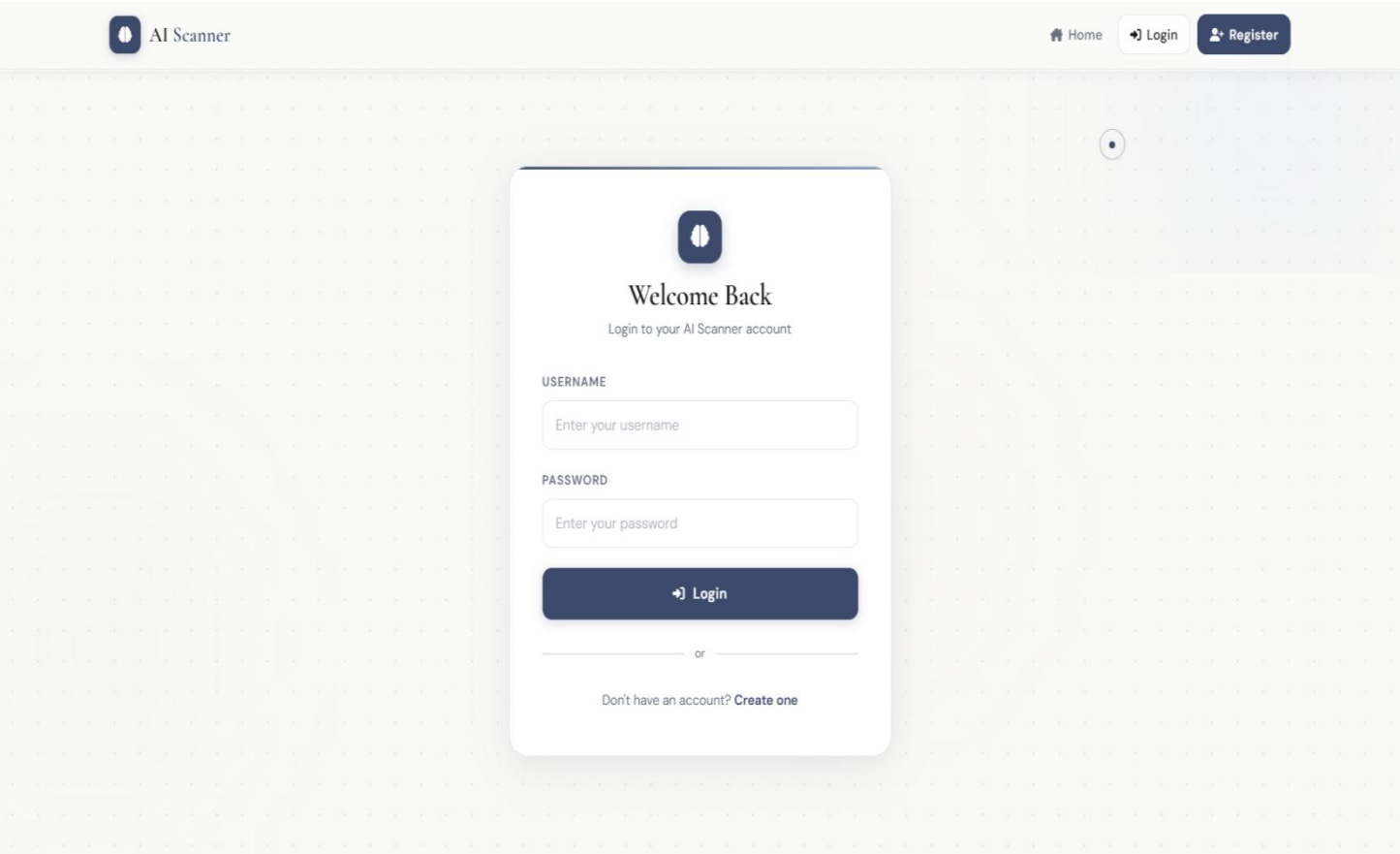
4.2.5 Dashboard



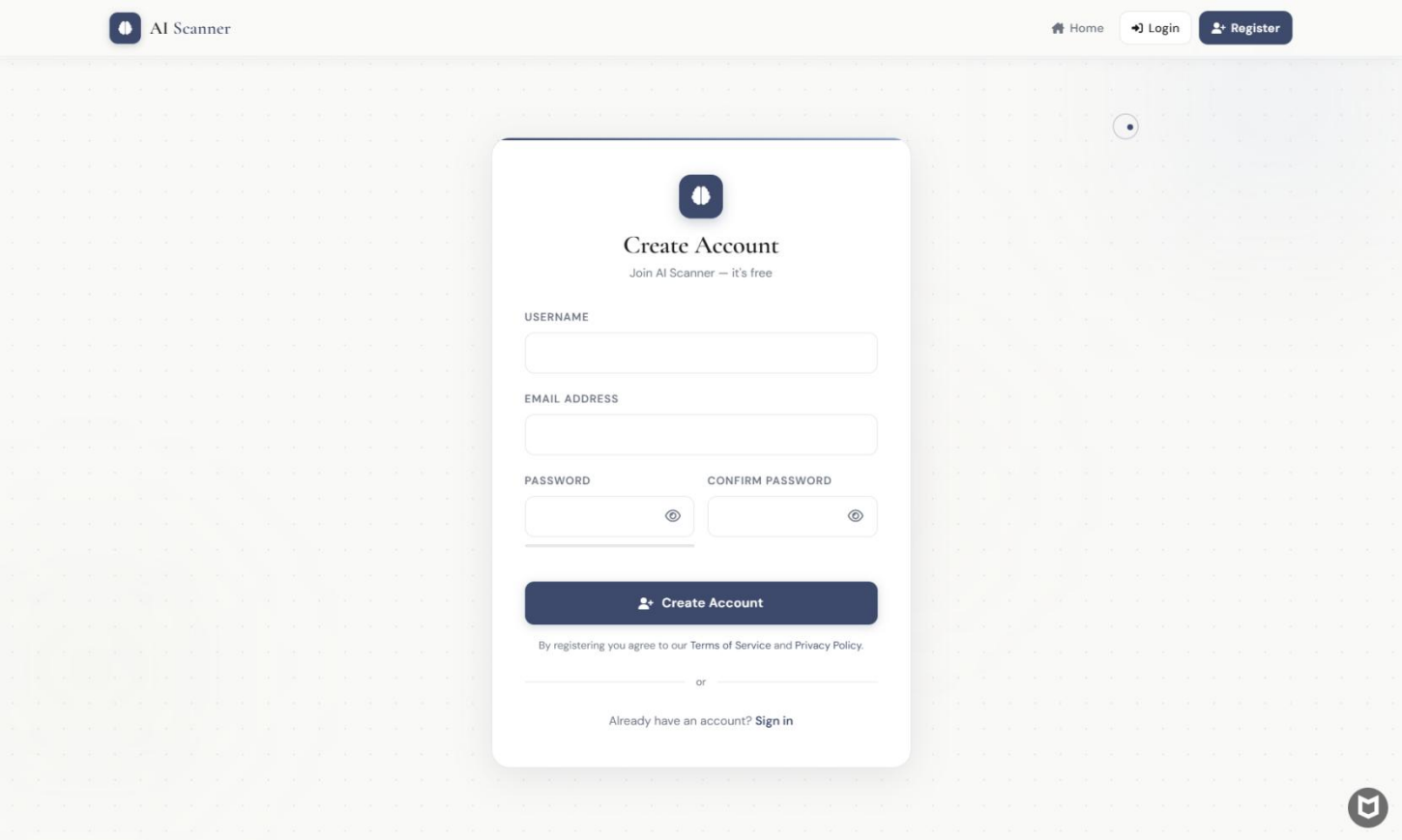
4.2.6 History


AI Scanner
Home
History
Dashboard
Logout
RECORDS
Scan History
Page 1 of 1
SCAN TYPE
FILE NAME
RESULT
ACCURACY
DATE
REPORT
DOWNLOAD
DELETE
text
User Input Text
Mixed
100.0%
April 3, 2026, 8:51 a.m.
View
PDF
Delete
image
images/IMG-20251129-WA...
Human
80.0%
April 3, 2026, 8:51 a.m.
View
PDF
Delete
document
DaxResume.pdf
Human
57.14%
April 3, 2026, 8:50 a.m.
View
PDF
Delete
Prev
Page 1 of 1
Next

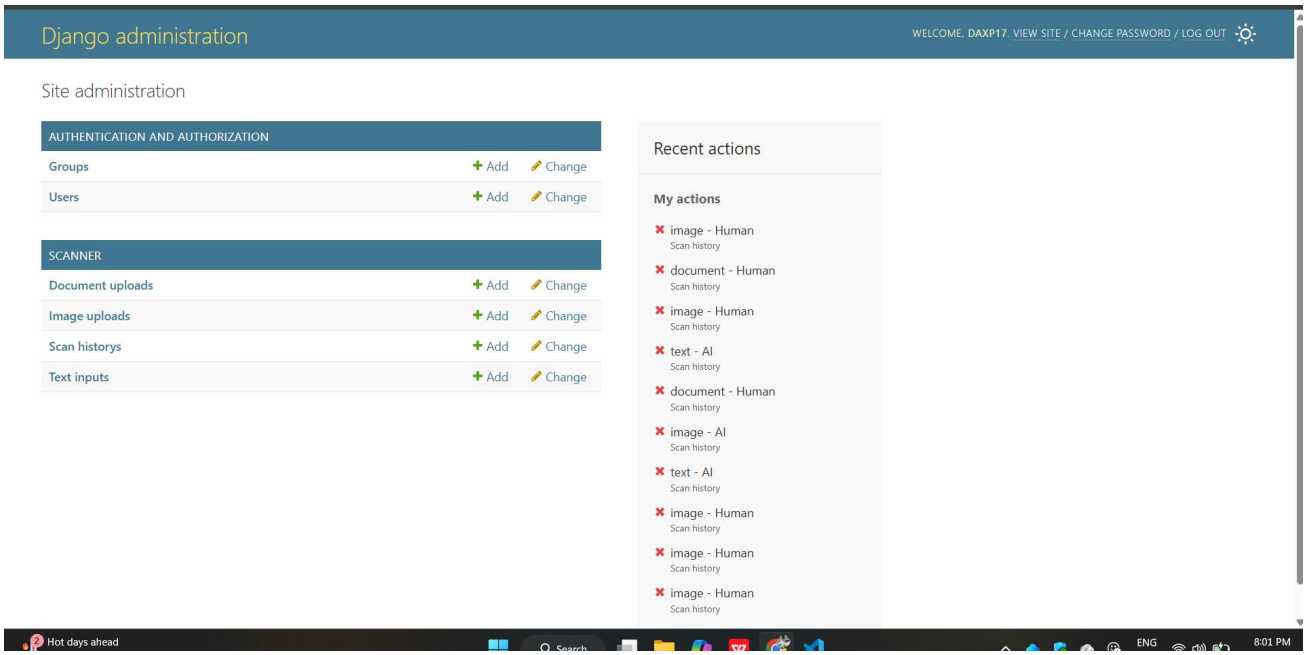
4.2.7 Login


AI Scanner
Home
Login
Register
Welcome Back
Login to your AI Scanner account
USERNAME
Enter your username
PASSWORD
Enter your password
Login
or
Don't have an account? Create one

4.2.8 Register



4.2.9 Admin panel



Chapter 5: Agile Documentation

5.1 Agile Project Charter

Project Charter			
Project Title:	AI-Based Document Authenticity Detection System		
Project Start:	18-12-2025	**Project End:**	10-04-2026
Business Need			
The increasing use of AI-generated content in academic and professional environments creates a need for reliable detection systems. This project aims to identify AI-generated text and images to ensure authenticity and prevent misuse.			
Project Scope			
The system includes a web-based application that allows users to upload documents and images. It performs AI detection using pretrained models for both text and images, provides probability-based results, and generates a detailed PDF report. The system also includes authentication and secure access control.			
Objective			
To develop an efficient and scalable system capable of detecting AI-generated text and images using machine learning techniques and providing clear, interpretable results.			
Milestone Schedule			
Milestone		**Target Completion Date**	**Actual Date**
Start Date		18-12-2025	20-01-2026
Analysis		21-01-2026	20-02-2026
Development		21-02-2026	30-03-2026
Testing & Deployment		31-03-2026	10-04-2026
Project Team			
Name	Panchal Dax Hareshkumar	**Project Guide**	Dr. Khyati Rami

5.2 Agile Roadmap / Schedule

NO	Time	Phase	Task
1	(Dec 2025 – Jan 2026	Initial Phase	Project initiation,topic finalization, and submission of project details.
2	Jan 2026	Planning Phase	Guide allocation and project definition approval.
3	Fab – March 2026	Development Phase	Implementation of core modules including text detection, image analysis, and system integration, along with regular weekly progress reporting.
4	March – April 2026	Evaluation Phase	Multiple presentations were conducted to review progress and improve the system based on feedback.
5	April 2026	Final Phase	Final documentation, report submission, and verification by the project guide.

5.3 Agile Project Plan

The project was developed using an incremental, Agile-inspired approach, where the system was divided into modules and built step-by-step. It started with project setup and user authentication, followed by a text detection module using the RoBERTa for sentence-level analysis. Then, an image detection module was implemented using CLIP along with OpenCV techniques. All modules were later integrated into a unified system with database support and report generation. Continuous testing and feedback-driven improvements ensured better performance and usability.

NO	Task	Priority	Start	End	Status
1	Requirement Analysis	High	01-01-26	30-01-26	Completed
2	UI Design	High	31-01-26	20-02-26	Completed
3	Backend Development	High	21-02-26	15-03-26	Completed
4	ML Integration	High	16-03-26	30-03-26	Completed
5	Testing	High	01-04-26	10-04-26	Completed

5.4 Agile User Story

No	Story Name	User Story
1	Document Analysis	As a user, I want to upload a document so that I can analyze whether the content is AI-generated or human-written.
2	Image Detection	As a user, I want to upload an image so that I can detect whether it is AI-generated or real.
3	Image Detection	As a user, I want to download the analysis report so that I can keep a record of the results securely.

5.5 Agile Release Plan

- **Release 1 – Basic Setup:**
  Implementation of project setup, user authentication (login/registration), and basic user interface.
- **Release 2 – Text Detection Module:**
  Integration of text processing and AI detection using a pretrained model (RoBERTa) with sentence-level classification.
- **Release 3 – Image Detection Module:**
  Implementation of image upload and AI detection using CLIP and OpenCV-based processing.
- **Release 4 – Integration & Report Generation:**
  Integration of all modules, database connectivity, and PDF report generation with secure access.
- **Release 5 – Testing & Final Deployment:**
  System testing, bug fixing, performance optimization, and final system deployment.

5.6 Agile Sprint Backlog

Sprint	Tasks
Sprint 1	Project setup, library installation, user registration and login implementation
Sprint 2	Document upload, text extraction, sentence tokenization
Sprint 3	Text AI detection using RoBERTa, probability calculation, result display
Sprint 4	Image upload, preprocessing, AI image detection using CLIP + OpenCV
Sprint 5	Database integration, report generation using ReportLab
Sprint 6	UI improvements, testing, bug fixing, and performance optimization

5.7 Agile Test Plan

Testing	Expected Result	Status
Unit Testing	Individual modules such as text analysis, image detection, and file upload were tested separately.	Success
Functional Testing	System features like document upload, AI detection, result display, and report generation were tested to ensure they work as expected.	Success
Integration Testing	All modules were tested together to verify smooth data flow from input to output.	Success
User Interface Testing	The UI was tested for usability, proper navigation, and correct display of results.	Success
Error Handling Testing	Tested system behavior for invalid inputs such as unsupported file formats, empty inputs, and oversized files.	Success

5.8 Earned-value and burn charts

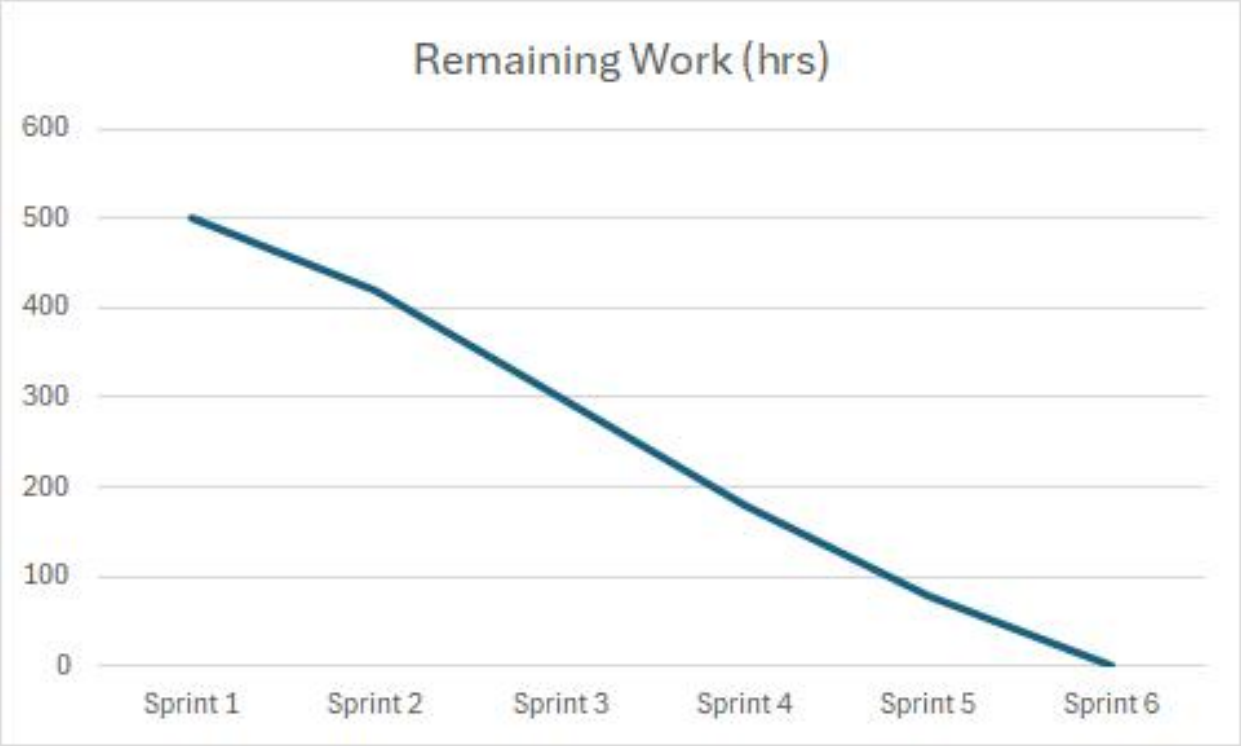
The project progress was evaluated using Earned Value Management (EVM) techniques. The key parameters used are:

- Planned Value (PV): Estimated effort for each module
- Earned Value (EV): Actual completed work
- Actual Cost (AC): Effort spent during implementation

5.8.1 EVM Table

No	Task	PV	EV	AC	SV	CV
1	UI Design	70	70	70	0	0
2	Backend Development	220	220	220	0	0
3	Text Detection (RoBERTa)	120	120	120	0	0
4	Image Detection (CLIP)	120	120	120	0	0
5	Testing & Integration	80	80	80	0	0

5.8.2 burn charts



Chepter 6: Proposed Enhancements

- Improve accuracy of AI detection by fine-tuning models like RoBERTa on domain-specific datasets
- Integrate advanced image detection models beyond CLIP for better classification
- Support larger file uploads using chunk-based processing instead of fixed size limits
- Add multilingual text detection capability for non-English content
- Enhance explainability by providing detailed reasoning for AI predictions
- Implement real-time detection for faster processing and user feedback
- Develop a dashboard for analytics and report history tracking
- Integrate cloud deployment for scalability and remote access

Chepter 7 : Conclusion

The AI-Based Document Authenticity Detection System was successfully designed and developed to identify AI-generated text and images. The system integrates pretrained models such as RoBERTa for text analysis and CLIP for image classification, along with OpenCV-based artifact analysis to improve detection reliability.

The application provides sentence-level classification, probability-based outputs, and visual highlighting, making the results more interpretable and user-friendly. Additionally, the system supports document upload, secure authentication, and automated PDF report generation.

The implementation demonstrates a hybrid approach, combining machine learning models with rule-based techniques, resulting in a practical and scalable solution for detecting AI-generated content.

Overall, the project meets its objectives by delivering a functional system capable of assisting users in verifying content authenticity efficiently.

Chepter 8: Bibliography

1.Liu, Y. et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach.

Available at: https://arxiv.org/abs/1907.11692

2. Radford, A. et al. (2021). Learning Transferable Visual Models From Natural Language Supervision (CLIP).

Available at: https://arxiv.org/abs/2103.00020

3. Hugging Face Documentation.

Available at: https://huggingface.co/docs

4. OpenCV Documentation.

Available at: https://opencv.org/

5. PyTorch Documentation.

Available at: https://pytorch.org/docs/

6. Python Software Foundation.

Available at: https://docs.python.org/3/

7. Django Documentation.

Available at: https://docs.djangoproject.com/

8. NLTK Documentation.

Available at: https://www.nltk.org/

9. PDFPlumber Documentation.

Available at: https://github.com/jsvine/pdfplumber

10. ReportLab Documentation.

Available at: https://www.reportlab.com/docs/

11. Chatgpt

Available at: https://chatgpt.com/

12.PlantUML Web Server

Available at: https://www.plantuml.com/plantuml/uml/